

Examen DeliverUS - Modelo Rider - Gestión de Pedidos por Repartidores

Enunciado del examen

Hasta ahora DeliverUS contaba con dos tipos de usuario: **customer** (cliente, que realiza pedidos) y **owner** (propietario, que gestiona sus restaurantes y confirma/envía/entrega los pedidos). En esta convocatoria se incorpora un **tercer tipo de usuario: rider** (repartidor).

¿Quién es el rider y qué puede hacer?

El rider es la persona que recoge en el restaurante los pedidos ya confirmados por el propietario y los entrega al cliente. Es un tipo de usuario independiente, con su propio registro, login y su propia aplicación frontend (**DeliverUS-Frontend-Rider**), igual que ya existen aplicaciones independientes para **owner** y **customer**.

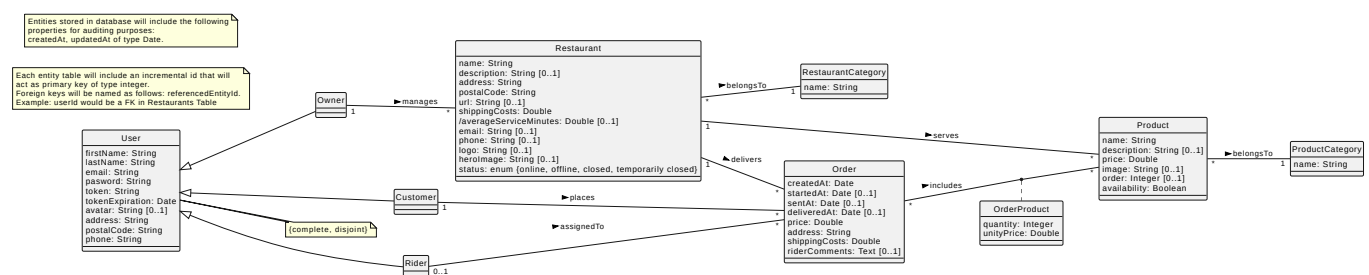
De forma resumida, un rider puede:

1. **Registrarse e identificarse** como rider (**POST /users/registerRider**, **POST /users/loginRider**). *Esta parte ya está implementada.*
2. **Consultar los pedidos disponibles:** aquellos pedidos que el restaurante ya ha confirmado (equivalente a que ya no están en estado **pending**) pero que todavía no ha reclamado ningún rider.
3. **Aceptar y recoger un pedido disponible:** el rider decide encargarse de ese pedido, que pasa a quedar asignado a él y a marcarse como recogido (**sentAt** se establece). A partir de ese momento deja de aparecer en el listado de disponibles para el resto de riders.
4. **Entregarlo al cliente:** una vez en reparto, el rider marca el pedido como entregado.
5. **Añadir comentarios de reparto:** el rider puede escribir una nota sobre la entrega (p. ej. "Dejado en la puerta, no contestan al timbre").
6. **Consultar sus propios pedidos** (los que tiene asignados), para hacer seguimiento de los que aún tiene pendientes de entregar y de los que ya completó.

⚠ Aceptar es el único paso de asignación del pedido. "Aceptar" (**accept**) reclama el pedido y marca **sentAt**, pasando el pedido directamente al estado **Enviado**.

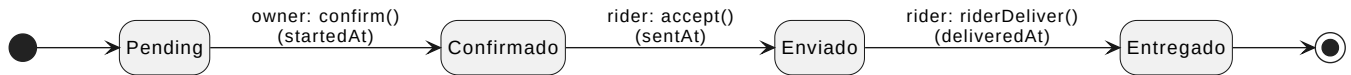
Modelado conceptual

Diagrama de clases de referencia de DeliverUS incluyendo las entidades y asociaciones a realizar en este examen.



El ciclo de vida de un pedido, ampliado

Ya conocía el ciclo de vida **pending** → **confirmado** (**startedAt**) → **enviado** (**sentAt**) → **entregado** (**deliveredAt**), gestionado por el owner mediante **confirm/send/deliver**. Con la incorporación del rider, los pasos de envío y entrega de un pedido los puede realizar también un rider (en lugar del owner), siguiendo el siguiente diagrama de estados:



Es necesaria la implementación de los siguientes requisitos funcionales:

Las pruebas de aceptación de cada RF se dividen en **Backend** (que se ayudan a verificar con **riderOrders.test.js**, que no debe modificarse) y **Frontend** (comportamiento que debe observarse al usar la aplicación: navegación tras una acción y mensajes de éxito/error).

RF1. Consultar pedidos disponibles. YA IMPLEMENTADO

Como rider,

quiero ver la lista de pedidos ya confirmados que ningún rider ha reclamado todavía

para poder elegir cuál voy a repartir.

Pruebas de aceptación — Backend (**GET /orders/available**):

- Sin sesión iniciada: **401 Unauthorized**.
- Con sesión iniciada como **customer** u **owner**: **403 Forbidden**.
- Con sesión iniciada como **rider**: **200 OK** y un array de pedidos.
- Un pedido en estado **pending** (sin **startedAt**, sin confirmar por el propietario) no debe aparecer en el listado.
- Un pedido **enviado** - es decir **sentAt** establecido - tampoco debe aparecer en el listado.
- Cada pedido devuelto debe incluir los datos resumidos del restaurante (**name**, **address**, **postalCode**) y del cliente (**firstName**).

Pruebas de aceptación — Frontend (**AvailableOrdersScreen.js**, ya implementada, para referencia):

- Al entrar en la pestaña "Available orders" se carga el listado automáticamente.
- Si la petición falla, se muestra un mensaje de error y se conserva el listado previo.

RF2. Aceptar un pedido

Como rider,

quiero reclamar un pedido disponible y marcarlo como recogido

para quedar asignado a él y empezar a repartirlo.

Pruebas de aceptación — Backend (**PATCH /orders/:orderId/accept**):

- Sin sesión iniciada: **401 Unauthorized**.
- Con sesión iniciada como **customer** u **owner**: **403 Forbidden**.

- Pedido inexistente: **404 Not Found**.
- Pedido en estado **pending** (sin confirmar): **409 Conflict**.
- Pedido ya asignado a rider: **409 Conflict**.
- Éxito: **200 OK**, y el pedido devuelto tiene **riderId** igual al id del rider autenticado y **sentAt** establecido con la fecha de aceptación.

Pruebas de aceptación — Frontend (**AvailableOrdersScreen.js**, ya implementada, para referencia):

- Al pulsar "Accept order" y recibir una respuesta correcta: se muestra un mensaje de éxito, se refresca el listado de disponibles (el pedido aceptado deja de aparecer) y se navega a la pestaña "My orders".
- Si la aceptación falla (p. ej. otro rider lo aceptó primero): se muestra un mensaje de error y se permanece en la pantalla de disponibles, sin navegar.

RF3. Consultar mis pedidos

Como rider,

quiero ver el listado de los pedidos que tengo asignados

para hacer seguimiento de mi trabajo pendiente y completado, y acceder fácilmente a los pedidos que debo entregar o comentar.

Pruebas de aceptación — Backend (**GET /orders/rider**):

- Sin sesión iniciada: **401 Unauthorized**.
- Con sesión iniciada como **customer** u **owner**: **403 Forbidden**.
- Con sesión iniciada como **rider**: **200 OK** y un array de pedidos.
- El listado solo debe incluir pedidos cuyo **riderId** sea el del rider autenticado (nunca pedidos de otros riders).
- Los pedidos pendientes de entregar (**deliveredAt** es **null**) deben listarse antes que los ya entregados.

Pruebas de aceptación — Frontend (pantalla "My Orders", **Ejercicio 4**):

- Cada pedido se muestra con el logo del restaurante, el id del pedido, el nombre del restaurante, los datos de entrega (cliente, teléfono, dirección) y el precio.
- Al pulsar sobre el **ImageCard** de cada pedido se navega a la pantalla **EditOrderCommentsScreen**.
- El listado se recarga al volver desde la pantalla de comentarios (**EditOrderCommentsScreen**), no solo la primera vez que se monta la pantalla.
- Si el rider no tiene pedidos asignados, se muestra un mensaje indicándolo en lugar de una lista vacía.
- Si la petición falla, se muestra un mensaje de error.

RF4. Entregar un pedido

Como rider,

quiero marcar un pedido como entregado al cliente

para reflejar que el reparto ha finalizado.

Pruebas de aceptación — Backend (**PATCH /orders/:orderId/riderDeliver**):

- Sin sesión iniciada: **401 Unauthorized**.
- Con sesión iniciada como **customer** u **owner**: **403 Forbidden**.
- Pedido inexistente: **404 Not Found**.
- Pedido asignado a otro rider: **403 Forbidden** (el rider autenticado no es el **riderId** del pedido).
- Pedido que no está en estado **Enviado** todavía, o ya entregado: **409 Conflict**.
- Éxito: **200 OK**, con **deliveredAt** establecido con la fecha de entrega.

Pruebas de aceptación — Frontend (pantalla "My Orders", Ejercicio 4):

- Un pedido aceptado pero no entregado muestra el botón "Confirm delivery" (color **brandGreen**).
- Al pulsarlo con éxito: se muestra un mensaje de éxito y se refresca el listado **en la misma pantalla**; el pedido actualizado deja de mostrar cualquier botón, al estar ya entregado.
- Si la acción falla: se muestra un mensaje de error y el listado no se modifica.

RF5. Comentar un pedido

Como rider,

quiero añadir o editar una nota sobre la entrega de un pedido que tengo asignado

para dejar constancia de incidencias (p. ej. dónde se dejó el pedido).

Pruebas de aceptación — Backend (**PATCH /orders/:orderId/riderComments**):

- Sin sesión iniciada: **401 Unauthorized**.
- Con sesión iniciada como **customer** u **owner**: **403 Forbidden**.
- Pedido asignado a otro rider: **403 Forbidden**.
- Pedido inexistente: **404 Not Found**.
- **riderComments** supera los 500 caracteres: **422 Unprocessable Entity**.
- **riderComments** vacío, **null** o ausente: válido, **200 OK** (el comentario es opcional).
- Éxito: **200 OK**, con el pedido devuelto reflejando el nuevo **riderComments**.

Pruebas de aceptación — Frontend (pantalla de comentarios, Ejercicio 4):

- Se accede a esta pantalla pulsando sobre la tarjeta de un pedido en el listado "My Orders".
- El formulario se inicializa con el comentario ya guardado del pedido (o vacío si no tenía).
- Si el usuario escribe más de 500 caracteres, la aplicación de frontend debe mostrar un error de validación **antes** de enviar la petición, y el botón de guardar no debe completar el envío. En cualquier caso, la aplicación de backend también debe asegurar que el comentario no supera los 500 caracteres y mostrar los errores enviados por el backend.
- Al guardar con éxito: se muestra un mensaje de éxito y se navega de vuelta al listado "My Orders" (**MyOrdersScreen**).
- Si guardar falla (p. ej. error del servidor o de conexión): se muestra un mensaje de error y se permanece en el formulario, sin perder lo escrito.

Parte Backend

Ejercicios (Backend)

1. Migraciones y Modelos (1 puntos)

Realice las modificaciones necesarias en migraciones y modelos para implementar el **RF2. Aceptar un pedido** y el **RF5. Comentar un pedido**

2. Enrutamiento, Middlewares y Validación (2 puntos)

En `src/routes/OrderRoutes.js` añada las rutas nuevas para implementar los siguientes requisitos funcionales:

- **RF2. Aceptar un pedido**
- **RF3. Consultar mis pedidos**
- **RF4. Entregar un pedido**
- **RF5. Comentar un pedido**

En `src/middlewares/OrderMiddleware.js`, implemente los siguientes middlewares nuevos:

- `checkOrderCanBeAccepted` para cumplir el **RF2. Aceptar un pedido**
- `checkOrderCanBeDeliveredByRider` para cumplir el **RF4. Entregar un pedido**
- `checkOrderIsAssignedToRider` para cumplir el **RF5. Comentar un pedido**

En `src/controllers/validation/OrderValidation.js`, añada las reglas de validación `updateRiderComments` necesarias para cumplir **RF5**.

Los middlewares `checkOrderVisible`, `checkOrderIsPending`, `checkOrderCanBeSent`, `checkOrderCanBeDelivered` y `checkOrderOwnership`, así como `handleValidation`, ya están implementados y puede tomarlos como referencia en caso necesario.

Para entregar (`riderDeliver`) un pedido **no es necesario escribir una función de controlador nueva**: reutilice la función de controlador `deliver` en `OrderController` que el owner ya usa para la misma transición de estado, enrutándolo también desde la nueva ruta del rider.

3. Controladores (2 puntos)

En `src/controllers/OrderController.js`:

- Implemente `accept` para cumplir con **RF2. Aceptar un pedido**.
- Implemente `indexRider` para cumplir con **RF3. Consultar mis pedidos**.
- Implemente `updateRiderComments` para cumplir con **RF5. Comentar un pedido**.

Para entregar (`riderDeliver`) un pedido **no es necesario escribir controladores nuevos**: reutilice el controlador `deliver` que el owner ya usa para la misma transición de estado, enrutándolo también desde la nueva ruta del rider.

Código Proporcionado (Backend)

Para este examen, se le entrega ya implementado:

1. Registro y login de riders (`registerRider/loginRider`, en `UserController.js` y `UserRoutes.js`).

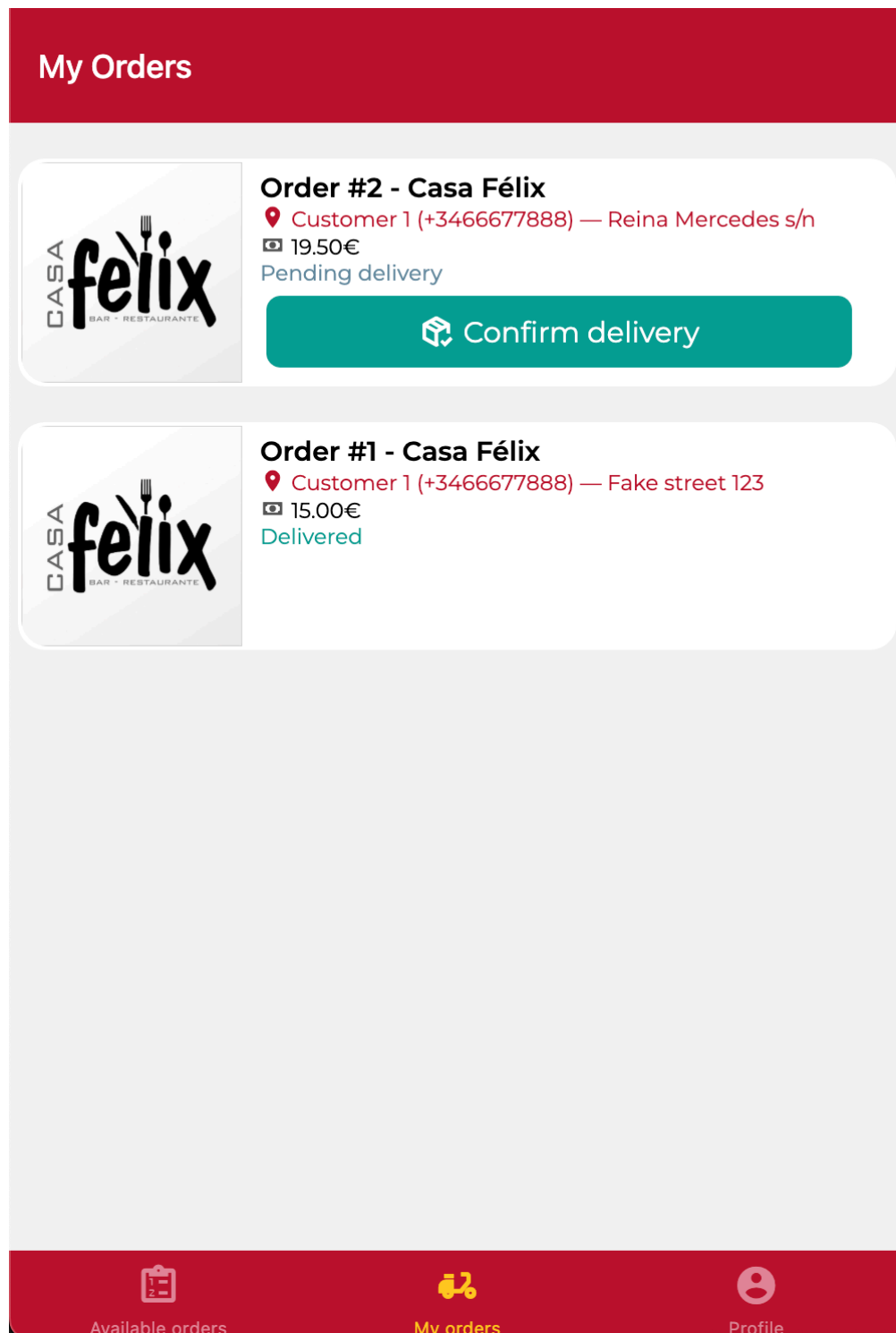
2. La comprobación de que un rider puede ver el detalle de cualquier pedido (rama `rider` de `checkOrderVisible`, en `OrderMiddleware.js`).
 3. Los controladores `confirm`, `send`, `deliver` y `show`, que podría reutilizar en caso necesario.
 4. El seeder de usuarios ya incluye un rider de pruebas: `rider1@rider.com / secret`.
 5. La función de controlador `findAvailableOrders` en `OrderController.js` como parte de la implementación del **RF1. Consultar pedidos disponibles**.
-

Parte Frontend (aplicación Rider)

Implemente las pantallas necesarias en `DeliverUS-Frontend-Rider` para que un rider pueda usar la aplicación. **NOTA: solo se valorará aquello que pueda ser utilizado tal y como lo haría un usuario final. No se valorarán pantallas que no renderizan o funcionalidades que no puedan probarse desde la interfaz**

Ejercicios (Frontend)

4. Pantalla de listado "My Orders" (2 puntos)



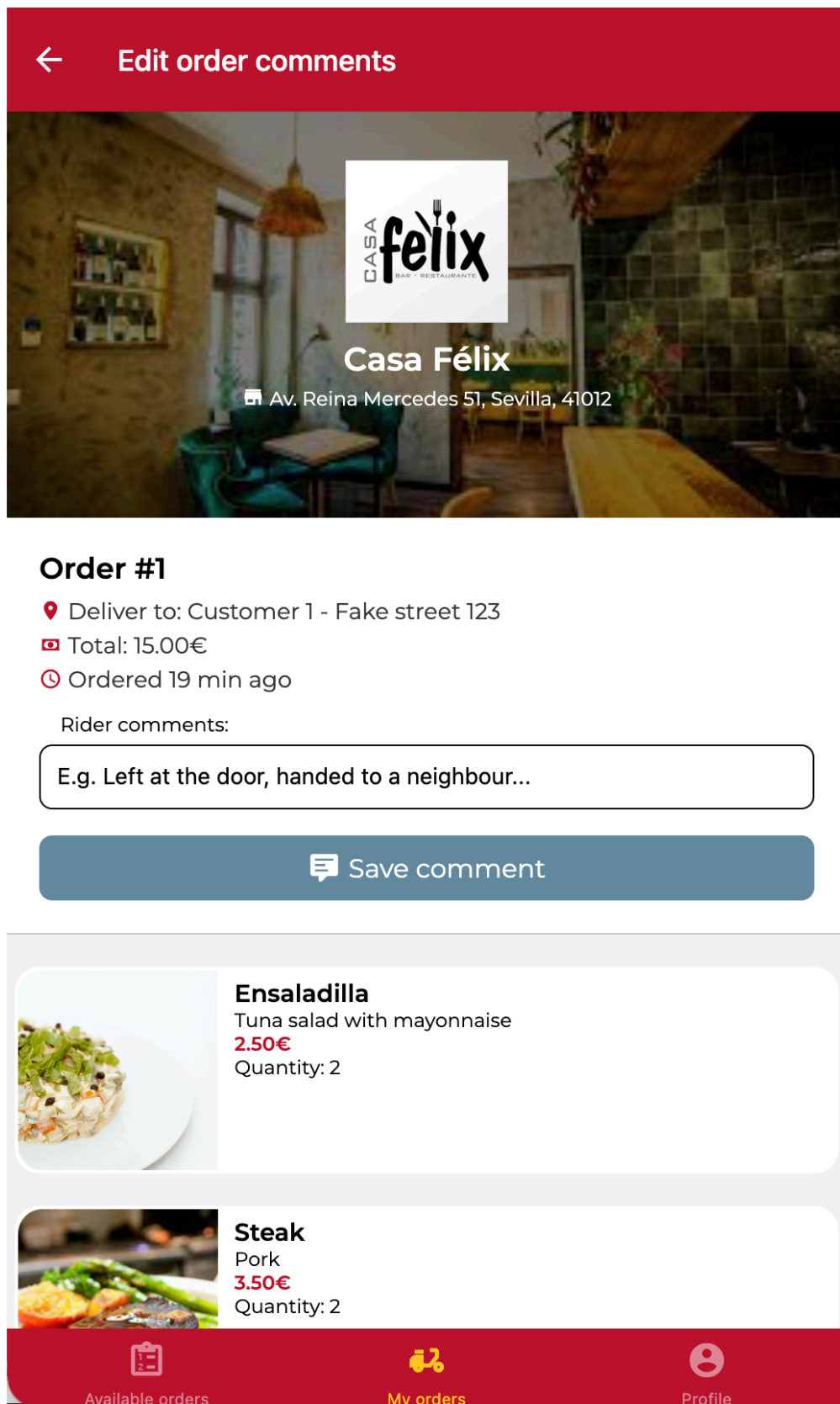
Pantalla: `src/screens/riderOrders/MyOrdersScreen.js`

Implemente esta pantalla para cumplir con **RF3. Consultar mis pedidos** y **RF4. Entregar un pedido**. Use el componente `ImageCard` para cada pedido. Puede utilizar `AvailableOrdersScreen.js` como inspiración.

API Backend necesaria (añada las funciones que falten en `src/api/OrderEndpoints.js`):

- `GET /orders/rider`
- `PATCH /orders/:orderId/riderDeliver`

5. Formulario de comentarios del pedido (2 puntos)



Pantalla: `src/screens/riderOrders/EditOrderCommentsScreen.js`

Ya dispone de una versión de esta pantalla donde se muestra una cabecera con datos del restaurante y del pedido, y listado de productos. Añada un formulario con **Formik** y un esquema de validación de **yup**, para cumplir con **RF5. Comentar un pedido** y muestre eventuales errores de validación enviados desde backend. Registre también esta pantalla en `MyOrdersStack.js` para que sea alcanzable desde `MyOrdersScreen` al pulsar sobre un pedido.

API Backend necesaria (añada la función que falte en `src/api/OrderEndpoints.js`):

- `PATCH /orders/:orderId/riderComments`

Fidelidad estética (1 punto)

Se valorará el grado de similitud visual de las interfaces entregadas con respecto a las capturas de pantalla proporcionadas.

Para ello, tenga también en cuenta lo siguiente:

- Use los colores corporativos definidos en `src/styles/GlobalStyles.js` (`brandBlue/brandBlueTap`, `brandGreen/brandGreenTap`, `brandPrimary`) y los iconos de `MaterialCommunityIcons` (paquete `@expo/vector-icons`) ya usados en el resto de la aplicación, manteniendo un estilo consistente con las pantallas ya existentes (`AvailableOrdersScreen.js`, `OrderDetailScreen.js`).
- Los iconos concretos de `MaterialCommunityIcons` a utilizar son:

Icono	Dónde
<code>package-variant-closed-check</code>	Botón "Confirm delivery" (Ejercicio 4).
<code>map-marker</code>	Dirección de entrega del cliente, en cada tarjeta de pedido (Ejercicio 4) y en la cabecera del pedido (Ejercicio 5).
<code>cash</code>	Precio del pedido, en cada tarjeta de pedido (Ejercicio 4) y en la cabecera del pedido (Ejercicio 5).
<code>comment-text</code>	Botón "Save comment" (Ejercicio 5).

Código Proporcionado (Frontend Rider)

Para este examen, se le entrega ya implementado:

1. Registro, login y perfil del rider (`LoginScreen.js`, `RegisterScreen.js`, `ProfileScreen.js` y su navegación).
2. La pantalla de pedidos disponibles (`AvailableOrdersScreen.js`), incluyendo el botón para aceptar (`acceptOrder`) un pedido.
3. Las funciones ya existentes en `src/api/OrderEndpoints.js`: `getAvailableOrders`, `getOrderDetail`, `acceptOrder`.
4. La estructura base (cabecera con datos del restaurante y listado de productos) de `EditOrderCommentsScreen.js`, a la que solo debe añadir el formulario.
5. El componente `InputItem`, ya preparado para integrarse con Formik.

Formato de petición/respuesta

Los códigos de estado y las reglas de negocio de cada endpoint están descritos en la RF correspondiente. Aquí solo se detalla la forma de los datos que no resulte obvia a partir de las RFs.

PATCH /orders/:orderId/riderComments (RF5)

Request:

```
{
  "riderComments": "Dejado en la puerta, no contestan al timbre"
}
```

Procedimiento de entrega

1. Borrar la carpeta **node_modules** de backend y de ambos frontend.
2. Crear un ZIP que incluya todo el proyecto. **Importante: Comprueba que el ZIP no es el mismo que te has descargado e incluye tu solución**
3. Avisa al profesor antes de entregar.
4. Cuando el profesor te dé el visto bueno, puedes subir el ZIP a la plataforma de Enseñanza Virtual. **Es muy importante esperar a que la plataforma te muestre un enlace al ZIP antes de pulsar el botón de enviar.** Se recomienda descargar ese ZIP para comprobar lo que se ha subido. Un vez realizada la comprobación, puedes enviar el examen.

Preparación del entorno

a) Windows

- Abra un terminal y ejecute el comando `npm run install:all:win`.

b) Linux/MacOS

- Abra un terminal y ejecute el comando `npm run install:all:bash`.

Ejecución

1. Backend

- Para **rehacer las migraciones y seeders**, abra un terminal y ejecute el comando

```
npm run migrate:backend
```

- Para **ejecutarlo**, abra un terminal y ejecute el comando

```
npm run start:backend
```

2. Frontend Rider

- Con el backend en ejecución, abra otro terminal y ejecute el comando

```
npm run start:frontend:rider
```

- Puede iniciar sesión con el usuario de pruebas `rider1@rider.com / secret`, o registrar un rider nuevo desde la propia aplicación.

Depuración

- Para **depurar el backend**, asegúrese de que **NO** existe una instancia en ejecución, pulse en el botón **Run and Debug** de la barra lateral, seleccione **Debug Backend** en la lista desplegable, y pulse el botón de *Play*.

Test

- Como ayuda puede ejecutar el conjunto de tests incluido `riderOrders.test.js`, que cubre el registro/login de riders y todo el ciclo de vida del pedido gestionado por el rider (disponibles, aceptar, entregar, comentarios y listado propio). Para ello ejecute el siguiente comando:

```
npm run test:backend
```

Advertencia: Los tests no pueden ser modificados.